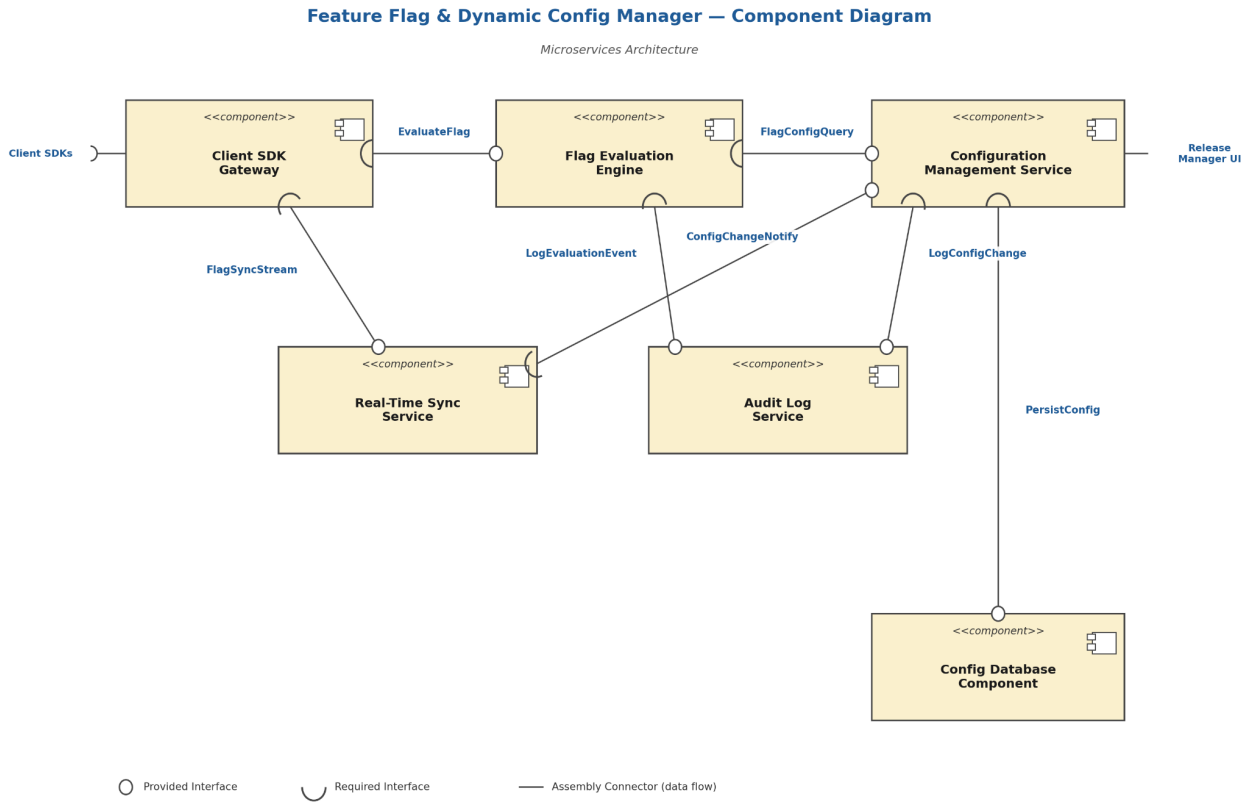
Name: Basava Prabhu
SRN: pes2ug24cs109

Software Engineering Lab3

Feature Flag & Dynamic Config Manager | Problem Statement #43

Component Modelling & Architectural Pattern Selection



Justification

Architecture Selection: We chose Microservices Architecture for the Feature Flag & Dynamic Config Manager System.

1. Architectural Choice

The Microservices Architecture was selected for this system. Under this style, each capability — flag evaluation, configuration management, real-time sync streaming, audit logging, and data persistence — is encapsulated in a separate, independently deployable service that communicates over well-defined interfaces (REST APIs, SSE/WebSocket channels, and internal data queries).

2. Two Scenario-Specific Reasons

Reason 1 — Independent Scalability of the Hot Evaluation Path

The Flag Evaluation Engine (NFR-001: p99 ≤ 15 ms under 500 RPS) must handle orders-of-magnitude more traffic than the Configuration Management Service, which is accessed only when a Release Manager updates a flag. Microservices allow the Evaluation Engine to be scaled horizontally in isolation — adding replicas only where load demands it — without scaling heavier services such as the Audit Log Service or the Config Database, preventing unnecessary resource waste and keeping evaluation latency predictable.

Reason 2 — Graceful Degradation Alignment with NFR-002

NFR-002 requires 99.9% uptime with SDK clients falling back to cached flag states during a control-plane outage. Microservices enable fault isolation: a failure in the Audit Log Service or the Config Management Service does not crash the Evaluation Engine. The Real-Time Sync Service can independently continue serving cached flag states to clients, satisfying the 'fail-safe' behaviour explicitly required by the scenario without coupling unrelated service lifecycles.

3. Security Advantage

Each microservice holds scoped credentials and exposes only the interfaces it needs to provide. The Config Database Component is exclusively accessed by the Configuration Management Service via a private internal interface (PersistConfig) — no other service can read or write flag configuration data directly. This principle of least-privilege access means a compromised Evaluation Engine cannot tamper with flag definitions or audit records, containing the blast radius of any breach to a single service boundary.

4. Performance Benefit

Separating the Flag Evaluation Engine from config-write and audit-logging concerns ensures the evaluation code path is lean and cache-optimised. The Engine can maintain an in-process memory cache of flag configurations (refreshed via ConfigChangeNotify events from the Real-Time Sync Service) so that the majority of evaluation requests never leave the process, achieving sub-millisecond lookups well within the 15 ms p99 budget. Asynchronous log writes to the Audit Log Service via the LogEvaluationEvent interface further ensure audit overhead is never on the critical path.

5. Component Summary

Component	Responsibility
Client SDK Gateway	Entry point; authenticates SDK keys; routes evaluation requests
Flag Evaluation Engine	Evaluates targeting rules & rollout buckets; returns boolean flag states in ≤15 ms
Configuration Management Svc	Manages flag definitions, rollout %, env overrides; exposes admin API

Component	Responsibility
Real-Time Sync Service	Pushes flag change events to connected SDKs via SSE/WebSocket within 2 s
Audit Log Service	Writes immutable, timestamped records of all config changes and evaluations
Config Database Component	Persistent store for all flag configurations; accessed only by Config Mgmt Svc